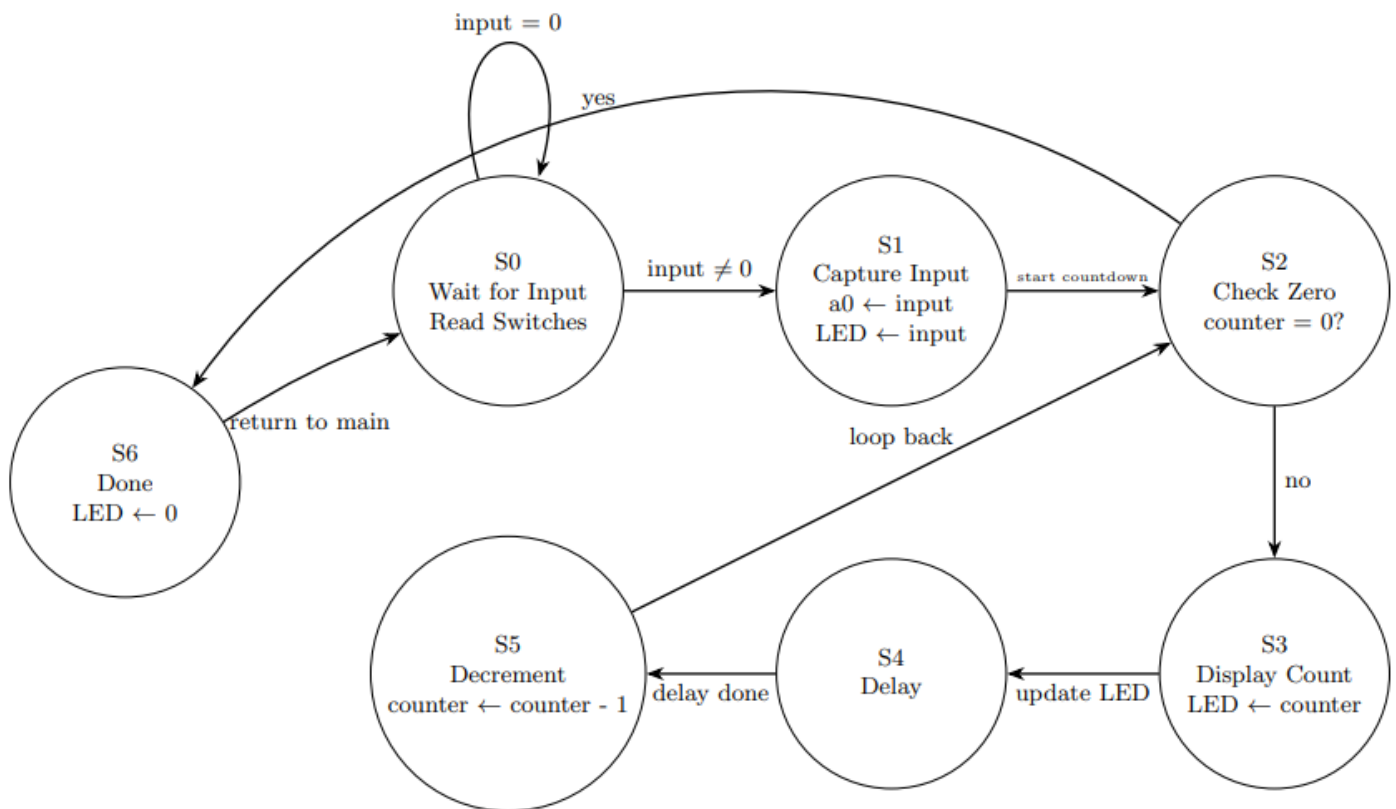


Syed Sualeh, Mohsin Mujtaba, Zaid Naqvi

Lab 10: State-Based Control Flow Using RISC-V Assembly

Task 1: FSM Design in Assembly

FSM control flow diagram for the assembly implementation



Task 2: Assembly Implementation of the FSM

Assembly source file implementing the FSM

Demonstration of correct stack usage in the code

```
.text
.globl main

main:
    li s1, 0x300    # Load Switch memory-mapped address
    li s2, 0x200    # Load LED memory-mapped address

wait_input:
    lw t0, 0(s1)    # Read switch values
    beq t0, zero, wait_input

    mv a0, t0       # Pass captured value to subroutine
    sw a0, 0(s2)    # Display initial value on LEDs

    jal ra, countdown # Use stack-based call

    j wait_input    # Return to input-waiting state

countdown:
    # Prologue: Save registers to stack
    addi sp, sp, -8
    sw ra, 4(sp)
    sw s0, 0(sp)

    mv s0, a0       # s0 holds the current count

count_loop:
    beq s0, zero, done # Exit when counter reaches zero
    sw s0, 0(s2)      # Update LED output with current count

    # Delay Loop (Adjust 'li' value based on FPGA clock frequency)
    li t1, 1000000   # Example delay constant
delay:
    addi t1, t1, -1
    bne t1, zero, delay

    addi s0, s0, -1   # Decrement counter
    j count_loop

done:
    sw zero, 0(s2)    # Clear LEDs

    # Epilogue: Restore registers from stack
    lw s0, 0(sp)
    lw ra, 4(sp)
    addi sp, sp, 8
    ret               # Return to main
```

Task 3: Instruction Memory Module Design

```

`timescale 1ns / 1ps

module instructionMemory #(
    parameter MEM_SIZE = 32 // We have 25 instructions (0 to 24)
)(
    input wire [31:0] instAddress,
    output reg [31:0] instruction
);

// Synchronous or Combinational ROM
always @(*) begin
    // The PC is byte-addressed (0, 4, 8...),
    // so we divide by 4 (shift right by 2) to get the array index.
    case (instAddress >> 2)
        // --- Main Program ---
        5'd0 : instruction = 32'h00000493; // li s1, 0x300
        5'd1 : instruction = 32'h00000913; // li s2, 0x200
        5'd2 : instruction = 32'h0004a283; // wait_input: lw t0, 0(s1)
        5'd3 : instruction = 32'h00028863; // beq t0, zero, wait_input
        5'd4 : instruction = 32'h00028513; // mv a0, t0
        5'd5 : instruction = 32'h00a92023; // sw a0, 0(s2)
        5'd6 : instruction = 32'h00c000ef; // jal ra, countdown
        5'd7 : instruction = 32'hff1ff06f; // j wait_input

        // --- Countdown Subroutine ---
        5'd8 : instruction = 32'hff810113; // countdown: addi sp, sp, -8
        5'd9 : instruction = 32'h00112223; // sw ra, 4(sp)
        5'd10 : instruction = 32'h00812023; // sw s0, 0(sp)
        5'd11 : instruction = 32'h00050413; // mv s0, a0
        // --- Loop ---
        5'd12 : instruction = 32'h00040a63; // count_loop: beq s0, zero, done
        5'd13 : instruction = 32'h00892023; // sw s0, 0(s2)
        5'd14 : instruction = 32'h000f4337; // li t1, 1000000 (part 1)
        5'd15 : instruction = 32'h24031304; // li t1, 1000000 (part 2)

        // --- Delay ---
        5'd16 : instruction = 32'hff130313; // delay: addi t1, t1, -1
        5'd17 : instruction = 32'hfe031ee3; // bne t1, zero, delay

        5'd18 : instruction = 32'hfff40413; // addi s0, s0, -1
        5'd19 : instruction = 32'hfeda8ae3; // j count_loop

        // --- Done / Return ---
        5'd20 : instruction = 32'h00092023; // done: sw zero, 0(s2)
        5'd21 : instruction = 32'h00012403; // lw s0, 0(sp)
        5'd22 : instruction = 32'h00412083; // lw ra, 4(sp)
        5'd23 : instruction = 32'h00810113; // addi sp, sp, 8
        5'd24 : instruction = 32'h00008067; // ret

        default: instruction = 32'h00000013; // nop (addi x0, x0, 0)
    endcase
end

endmodule

```



Memory initialization file containing assembly instructions

```
// Lab 10: FSM Countdown Instruction Memory
// Format: Hexadecimal Machine Code
00000493 // @0: li s1, 0x300
00000913 // @4: li s2, 0x200
0004a283 // @8: wait_input: lw t0, 0(s1)
00028863 // @C: beq t0, zero, wait_input
00028513 // @10: mv a0, t0
00a92023 // @14: sw a0, 0(s2)
00c000ef // @18: jal ra, countdown
ff1ff06f // @1C: j wait_input
ff810113 // @20: countdown: addi sp, sp, -8
00112223 // @24: sw ra, 4(sp)
00812023 // @28: sw s0, 0(sp)
00050413 // @2C: mv s0, a0
00040a63 // @30: count_loop: beq s0, zero, done
00892023 // @34: sw s0, 0(s2)
000f4337 // @38: li t1, 1000000 (part 1: lui)
24031304 // @3C: li t1, 1000000 (part 2: addi)
ff130313 // @40: delay: addi t1, t1, -1
fe031ee3 // @44: bne t1, zero, delay
fff40413 // @48: addi s0, s0, -1
feda8ae3 // @4C: j count_loop
00092023 // @50: done: sw zero, 0(s2)
00012403 // @54: lw s0, 0(sp)
00412083 // @58: lw ra, 4(sp)
00810113 // @5C: addi sp, sp, 8
00008067 // @60: ret
```



Assessment Rubric

Lab 10: State-Based Control Flow Using RISC-V Assembly

Name:	Student ID:	Section:
-------	-------------	----------

Points Distribution

	Task No.	LR 2 (Code)	LR 5 (Results)	LR 11 (Design)
In-Lab	Task 1	-	-	/20
	Task 2	/20	/20	-
	Task 3	/20	-	-
Total Points		/40	/20	/20
CLO Mapped		CLO 1		

Affective Domain Rubric		Points	CLO Mapped
AR7	Report Submission & Git Upload	/10 & /10	CLO 1

CLO	Total Points	Points Obtained
1	100	
Total	100	

For description of different levels of the mapped rubrics, please refer to the Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics provided here.